

Divide and Conquer

Polynomial Multiplication:

$$A(x) = a_0 + a_1x + \cdots + a_{d-1}x^{d-1}$$

$$B(x) = b_0 + b_1x + \cdots + b_{d-1}x^{d-1}$$

$$C = A \cdot B$$



$$C(x) = c_0 + c_1x + \cdots + c_{2d-2}x^{2d-2}$$

Space & time n^2 ?

Degree of the
resulting polynomial

All of these are polynomials with degree
less than some n . [$< n \leftarrow (2d - 1)$]

$$c_0 = a_0b_0, c_1 = a_1b_0 + b_1a_0, c_k = \sum_{j=0}^k a_jb_{k-j}$$

Space n & time n^2 ?

$$a_j = b_j = 0, j > d - 1$$

Like integer multiplication!

$$\alpha \leftarrow 3 \ 7 \ 9 \ 1 \longrightarrow 1 + 9x + 7x^2 + 3x^3 = A$$

$$\beta \begin{array}{l} \times \\ \swarrow \\ 1 \ 2 \ 5 \longrightarrow 5 + 2x + 1x^2 + 0x^3 = B \\ \hline \end{array}$$

$$4 \ 7 \ 3 \ 8 \ 7 \ 5 \qquad \alpha \times \beta = C(10), C(x) = A(x) \times B(x)$$

$$A \times B = 5 + (45 + 2)x + (35 + 18 + 1)x^2 + (15 + 14 + 9)x^3 + (6 + 7)x^4 + 3x^5$$



At $x = 10$:

$$5 + 47 \times 10 + 54 \times 100 + 38 \times 1000 + 13 \times 10000 + 3 \times 100000 = 473875$$

Algorithm 1:

$$c_k = \sum_{j=0}^k a_j b_{k-j}$$

Two nested for loops with the outer one running from 0:n-1.

$$O(n^2)$$

Also counting # of products..... $\Theta(n^2)$

Consider all $c_k = \sum_{j=0}^k a_j b_{k-j}, k > \frac{n}{2}$

At least $\frac{n}{2}$ Multiplications, $\frac{n}{2}$ times

Complexity... $\geq \frac{n^2}{4}$

So algorithm 1 is both $O(n^2)$ & $\Omega(n^2)$

Algorithm 2:

Karatsuba!

$$A(x) = \boxed{a_0 + a_1x + \dots + a_{n-1}x^{n-1}}$$

$$B(x) = \boxed{b_0 + b_1x + \dots + b_{n-1}x^{n-1}}$$

$$[A \cdot B](x)$$

$$= [A_l \cdot B_l](x) + ([A_l \cdot B_h](x) + [A_h \cdot B_l](x))x^{n/2} + [A_h \cdot B_h](x)x^n$$

Not evaluating the poly! $O(n^{\log_2 3})!$

Algorithm 3: **Fast Fourier transform** can make it even more faster!

‘Interpolation’:

Any degree $< N$ polynomial is determined by its evaluation on N distinct point.

$x_0, x_1, \dots, x_{N-1}$ are the distinct points

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{N-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{N-1} & x_{N-1}^2 & \cdots & x_{N-1}^{N-1} \end{bmatrix} \begin{bmatrix} C_0 \\ \vdots \\ C_{N-1} \end{bmatrix} = \begin{bmatrix} y_0 \\ \vdots \\ y_{N-1} \end{bmatrix}$$

Can't be all 0s

$$VC = Y, C = V^{-1}Y$$

Invertible as:

$V_{i,j} = x_i^j; i, j \in \{0, 1, \dots, N-1\} \rightarrow$ Vandermonde matrix

Square

$$\det(V) = \prod_{0 \leq i_2 < i_1 \leq N-1} (x_{i_1} - x_{i_2})$$

So...

Polynomial multiplication

Input: Coefficients of two polynomials, $A(x)$ and $B(x)$, of degree d

Output: Their product $C = A \cdot B$

Selection

Pick some points $x_0, x_1, \dots, x_{n-1}$, where $n = 2d + 1$

Evaluation

Compute $A(x_0), A(x_1), \dots, A(x_{n-1})$ and $B(x_0), B(x_1), \dots, B(x_{n-1})$

Multiplication

Compute $C(x_k) = A(x_k)B(x_k)$ for all $k = 0, \dots, n - 1$

Interpolation

Recover $C(x) = c_0 + c_1x + \dots + c_{2d}x^{2d}$

→ Gaussian elimination of augmented matrix.

Laplace expansion – $n!$

Interpolation is n^3 complexity?! Actually n^2 , as Vandermonde is special*.

AND evaluation will incur n^2 complexity &
multiplication (base step) will incur n complexity

An evaluation involves n multiplication and $n - 1$ additions.

FFT to rescue

*<https://www.sciencedirect.com/science/article/pii/S0885064X97904428>

The Fast Polynomial Multiplication Algorithm:

Input: $A(x), B(x)$

Output: $C = A \cdot B$

Selection:

$$\omega_N^k = \exp\left(-\frac{2\pi i}{N}k\right), k = 0, 1, \dots, N-1 \quad O(N)$$

Evaluation:

Coefficients
↑

$$A(\omega_N^k), B(\omega_N^k), \forall k \leftarrow \mathbf{F}\mathbf{c}_A, \mathbf{F}\mathbf{c}_B \quad O(N \log N)$$

Multiplication:

$$C(\omega_N^k) = A(\omega_N^k)B(\omega_N^k), \forall k \quad O(N)$$

Interpolation:

↘

$$\mathbf{c}_C = \frac{1}{N} (\mathbf{F}\mathbf{C}^*)^* \quad O(N \log N)$$

Graphs

Definition:

$$G = (V, E), E \subseteq V \times V$$

Undirected -

$$(u, v) \in E \leftrightarrow (v, u) \in E \dots \dots \{u, v\} \in E$$

Directed -

$(u, v) \in E$ does not imply $(v, u) \in E$ & vice – versa

Representation:

Adjacency Matrix

$$n = |V|$$

Vertices: $v_1, \dots, v_n$

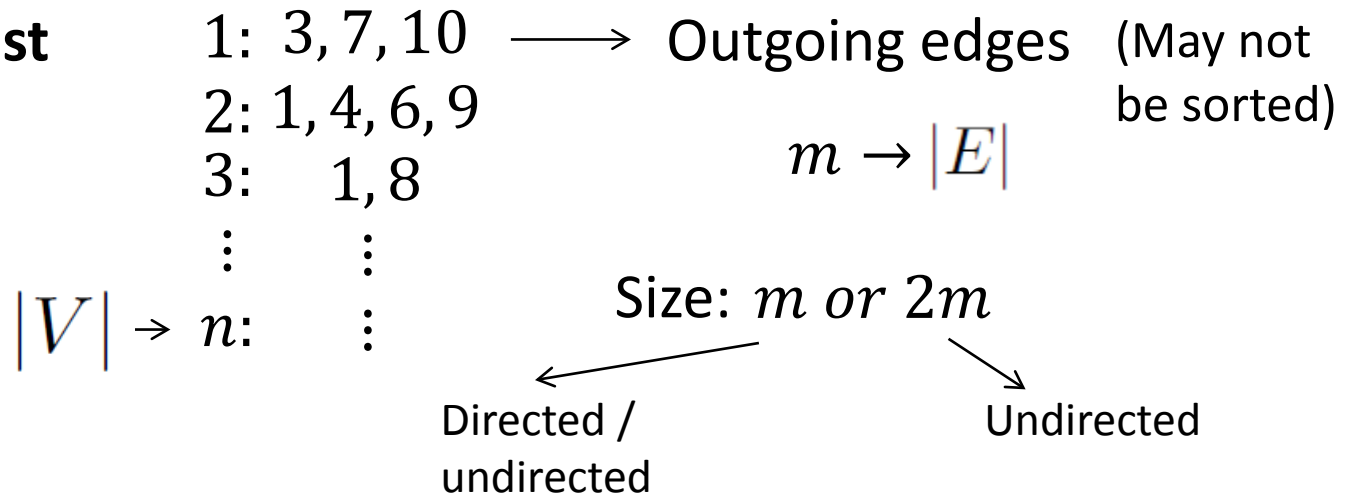
$n \times n$ array

$$a_{ij} = \begin{cases} 1 & \text{if there is an edge from } v_i \text{ to } v_j \\ 0 & \text{otherwise.} \end{cases}$$

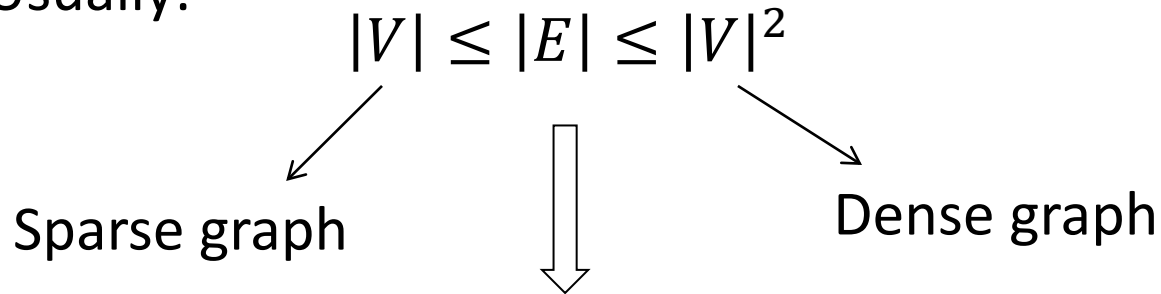
Symmetric for undirected graphs

Presence of an edge
in constant time!

Adjacency List



Usually:



Crucial criteria for selecting graph algorithm!

WWW, Social Network – Sparse graph with billions of vertices (Adjacency list?)

Operations	Matrix	List
Search $(u, v) \in E$	$O(1)$	$O(n)/O(d)$
Find $N^+(v), v \in V$	$O(n)$	$O(1)[O(n)/O(d)]$
Space	$O(n^2)$	$O(m)$

Max number of neighbors a vertex can have. d is the maximum degree of a vertex in the graph.

Returning the pointer or the entire list?

A vertex u is a *neighbor* of (or equivalently *adjacent* to) a vertex v in a graph $G = (V, E)$ if there is an edge $\{u, v\} \in E$. For a directed graph we use the terms *in-neighbor* if $(u, v) \in E$ and *out-neighbor* if $(v, u) \in E$.

The *degree* $d_G(v)$ of a vertex $v \in V$ in a graph $G = (V, E)$ is the size of the neighborhood $(|N_G(v)|)$. For directed graphs we use *in-degree* ($d_G^-(v) = |N_G^-(v)|$) and *out-degree* ($d_G^+(v) = |N_G^+(v)|$).